

Optimizing neural network architectures using Genetic Algorithms, Particle Swarm Optimization, and Simulated Annealing

Joshua Durrant Jan Hausding Leander Hemmi

Department of Computer Science (D-INFK)
ETH Zurich

Optimization Methods for Engineers, August 2025

Overview

- 1 Introduction
- 2 Algorithms
- 3 Parametric Optimization
- 4 Results
- 5 Conclusion and Discussion
- 6 Future Work

Introduction - Problem Description

- **Given** an image-based dataset with n data points $d_1, d_2, \dots, d_n \in \mathbb{R}^{H \times W \times C}$, where H, W, C represent height, width and channels respectively, and $n \in \mathbb{N}$ represents the number of samples.
- **Find** $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)^\top \in \mathcal{D}^n$ which maximizes

$$f(\alpha, \mathcal{D}) := \lambda_2 \cdot \text{Accuracy}(\alpha, \mathcal{D}) - \lambda_1 \cdot \text{NetworkSize}(\alpha)$$

Where:

- α represents the network architecture parameters
- $\mathcal{D}$ is the dataset
- λ_1, λ_2 are weighting factors balancing size vs. accuracy
- $\text{NetworkSize}(\alpha)$ counts the total number of parameters
- $\text{Accuracy}(\alpha, \mathcal{D})$ is the classification accuracy on dataset $\mathcal{D}$

- **Motivation** (Optimization of NNA): Please write :) [1]

Introduction - Architecture Components

Unlike traditional approaches that only focus on fully connected layers, we extend the search space to include the following layers:

- **Convolutional Layers (Conv1D/Conv2D):** Feature extraction with learnable filters
- **Fully Connected Layers (FC):** Dense layers for classification
- **Batch Normalization:** Normalization of the values for training stability
- **Max Pooling:** Spatial dimension reduction
- **Dropout:** Regularization technique to avoid overfitting
- **Activation Functions:** ReLU, Tanh, Sigmoid

Introduction - Solution Space Characteristics

- Non-linear cost function
 $f(\alpha, \mathcal{D}) := \lambda_2 \cdot \text{Accuracy}(\alpha, \mathcal{D}) - \lambda_1 \cdot \text{NetworkSize}(\alpha)$ combines discrete network size with continuous accuracy metrics
- Mixed discrete-continuous solution space consisting of architecture topology (discrete) and hyperparameters (continuous)
- Solution space can be represented by encoding each architecture α as a variable-length chromosome
- **Naive Approach:** Exhaustive search with runtime $\mathcal{O}(C^L)$ for L max layers with C component types becomes intractable for realistic network sizes

Introduction - Project Objectives

- Investigate how **Genetic Algorithms (GA)**, **Particle Swarm Optimization (PSO)**, and **Simulated Annealing (SA)** perform when applied to the problem.
- Balance the trade-off between network size and accuracy
- Evaluate the effectiveness of different layer types in automated architecture design
- Assess the performance of these algorithms by comparing them to baseline methods, such as **Random Search (RS)** and **Grid Search (GS)**.
- Assess the performance of the algorithms across different dataset complexities

Given a image-based dataset $\mathcal{D} = \{d_1, \dots, d_n\}$

Random Search (RS)

Cost function: $f(\alpha^{(i)}) = \lambda_2 \cdot \text{Accuracy}(\alpha^{(i)}, \mathcal{D}) - \lambda_1 \cdot \text{NetworkSize}(\alpha^{(i)})$

- 1 Generate N random candidate architectures $\alpha^{(1)}, \dots, \alpha^{(N)}$
- 2 Train each network and evaluate $f(\alpha^{(i)})$ for $i = 1, 2, \dots, N$
- 3 Return $(\alpha^{(best, RS)}, f(\alpha^{(best, RS)}))$ with $f(\alpha^{(best, RS)}) \geq f(\alpha^{(i)}), \forall \alpha^{(i)}$

Grid Search (GS)

Cost function: $f(\alpha^{(i)}) = \lambda_2 \cdot \text{Accuracy}(\alpha^{(i)}, \mathcal{D}) - \lambda_1 \cdot \text{NetworkSize}(\alpha^{(i)})$

- 1 Define systematic grid over architecture hyperparameters
- 2 Evaluate $f(\alpha^{(i)})$ for each grid point $\alpha^{(i)}$
- 3 Return $(\alpha^{(best, GS)}, f(\alpha^{(best, GS)}))$ with maximal fitness

Algorithms - Genetic Algorithm

Genetic Algorithm (GA) [1]:

Fitness function: $fitness = \lambda_2 \cdot Accuracy(\alpha^{(i)}, \mathcal{D}) - \lambda_1 \cdot NetworkSize(\alpha^{(i)})$

Encoding: Variable-length chromosomes representing layer sequences

- Gene type: {Conv2D, Conv1D, FC, BatchNorm, MaxPool, Dropout, Activation}
- Gene parameters: kernel_size, stride, filters, units, etc.

Algorithm Steps:

- 1 Create population_size random architectures $\alpha^{(i)} = (layer_1, layer_2, \dots, layer_k)$
- 2 Train networks and compute fitness scores
- 3 Tournament selection with tournament_size individuals
- 4 Single-point crossover with probability crossover_prob, preserving valid layer sequences
- 5 Add/remove layers with probability mutation_add_prob / mutation_del_prob and modify layer parameters with probability mutation_param_prob
- 6 Generate new population, repeat for num_generations generations
- 7 Return $(\alpha^{(best, GA)}, fitness^{(best, GA)})$

Algorithms - Particle Swarm Optimization

Particle Swarm Optimization (PSO) [2], [3]:

Position Representation: Continuous vector $x^{(i)} \in \mathbb{R}^d$ encoding the architecture parameters

Velocity Update:

$$v^{(i)}(t+1) = w \cdot v^{(i)}(t) + c_1 \cdot r_1 \cdot (pbest^{(i)} - x^{(i)}(t)) + c_2 \cdot r_2 \cdot (gbest - x^{(i)}(t))$$

Position Update: $x^{(i)}(t+1) = x^{(i)}(t) + v^{(i)}(t+1)$

Architecture Decoding: Map continuous positions to discrete architectures:

- Layer count: $\lfloor x_1 \cdot \text{max_layers} \rfloor$
- Layer types: $\text{argmax}(\text{softmax}([x_2, x_3, \dots, x_7]))$
- Layer parameters: sigmoid/tanh transformations to valid ranges

Algorithm Steps:

- 1 Initialize swarm: Set positions $x^{(i)}(0)$ at random and set velocities $v^{(i)}(0) = 0$
- 2 Convert positions to architectures and train the networks
- 3 Update personal best $pbest^{(i)}$ and global best $gbest$
- 4 Update velocities and positions
- 5 Repeat for `num_iterations` iterations
- 6 Return $(\alpha^{(best, PSO)}, fitness^{(best, PSO)})$

Algorithms - Simulated Annealing

Simulated Annealing (SA) [4]:

State Representation: Current architecture $\alpha_{current}$

Neighbourhood Function: Small architectural modifications:

- Add/remove single layer
- Modify layer parameters
- Swap adjacent layers

Algorithm Steps:

- 1 Start with random architecture $\alpha_{current}$
- 2 Apply random modification to generate $\alpha_{neighbour}$
- 3 Acceptance Criterion:
 - If $f(\alpha_{neighbour}) < f(\alpha_{current})$: accept
 - Else: Accept with probability $e^{-\left(\frac{f(\alpha_{neighbour}) - f(\alpha_{current})}{T}\right)}$
- 4 Reduce temperature $T = T \cdot \text{cooling_rate}$ every 10 iterations
- 5 Repeat until convergence or `max_iterations`
- 6 Return $(\alpha^{(best, SA)}, fitness^{(best, SA)})$

Parametric Optimization - Variable Parameters

All algorithms run on variable parameters:

- Variable parameters for GA:

pop_size	cross_prob	mut_add	mut_del	mut_param	tourn_size	n_gen
Population size	Crossover prob.	Add mutation	Del mutation	Param mutation	Tournament size	Generations

- Variable parameters for PSO:

swarm_size	w	c ₁	c ₂	num_iter
Swarm size	Inertia weight	Cognitive coefficient	Social coefficient	Iterations

- Variable parameters for SA:

initial_temp	cooling_rate	max_iter
Initial temperature	Cooling rate	Max. iterations

- Other degrees of freedom include: the type of optimizer used in the neural network, learning rate used in training, type of crossover used in the GA, choice of r_1 and r_2 , etc.

Parametric Optimization - Fixed Parameters

- General fixed parameters used in this work:
 - $\lambda_1 = 10^{-6}$ (parameter penalty)
 - $\lambda_2 = 1.0$ (accuracy weight)
 - `max_layers` = 8
 - `epochs` = 3 (training) and 20 (evaluation)
 - `learning_rate` = 0.001
 - `optimizer` = Adam with `weight_decay` = 10^{-4}
- Algorithm-specific fixed parameters used in this work:
 - GA: `pop_size` = 40, `n_gen` = 8
 - PSO: `swarm_size` = 40, `num_iter` = 30
 - SA: `max_iter` = 1200

Simple algorithm to find the best parameters:

```
for each algorithm  $\in \{\text{GA, PSO, SA}\}$  do  
  for each parameter_set  $\in$  parameter_combinations do  
    for  $k = 1$  to 3 do  
      Split dataset into train/validation  
      Run algorithm with parameter_set  
      Collect: best_fitness, convergence_time, final_accuracy  
    end for  
    Compute: avg_fitness, avg_time, avg_accuracy  
  end for  
  Select parameter_set with best avg_fitness  
end for
```

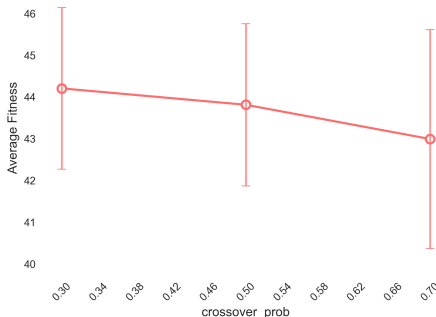
Dataset (CIFAR10) Splits:

- Training: 60% (Architecture search)
- Validation: 20% (Fitness evaluation)
- Test: 20% (Final performance assessment)

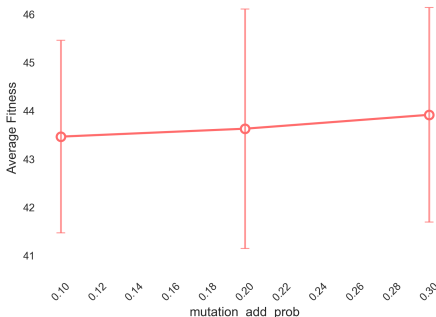
Parametric Optimization - Genetic Algorithm

GA Parameter Sensitivity Analysis

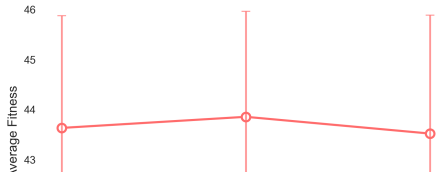
Average Fitness vs. crossover_prob



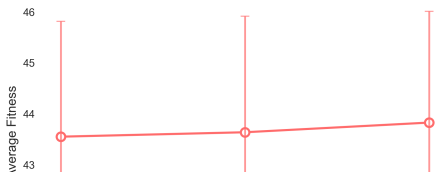
Average Fitness vs. mutation_add_prob



Average Fitness vs. mutation_del_prob



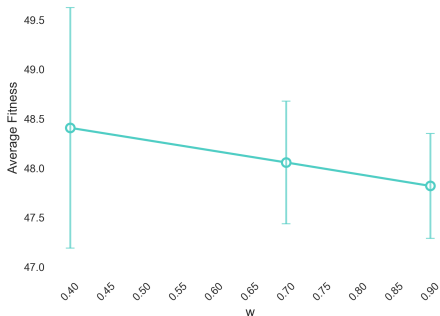
Average Fitness vs. mutation_param_prob



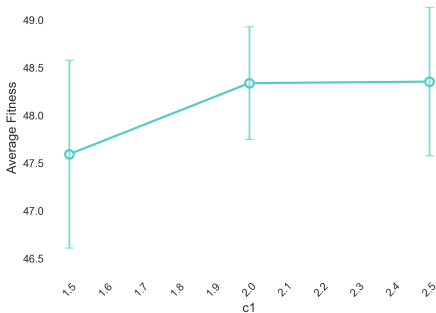
Parametric Optimization - Particle Swarm Optimization

PSO Parameter Sensitivity Analysis

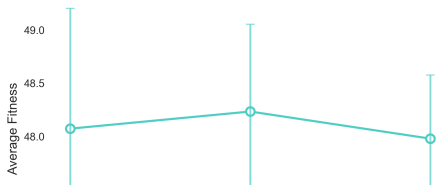
Average Fitness vs. w



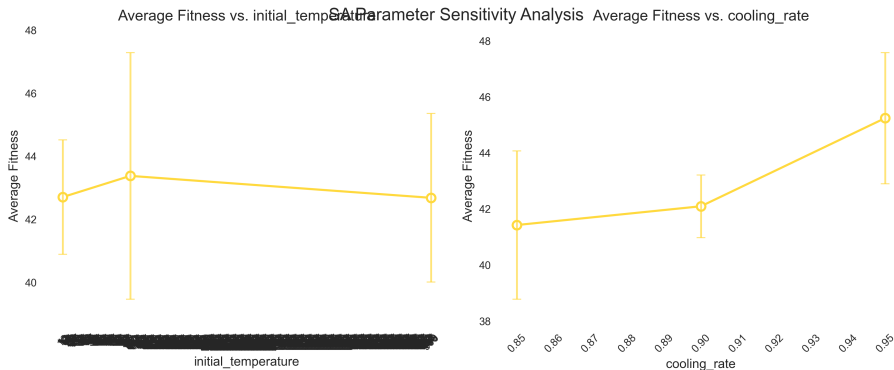
Average Fitness vs. $c1$



Average Fitness vs. $c2$



Parametric Optimization - Simulated Annealing



Results

Conclusion and Discussion

PLEASE WRITE

Future Work and Limitations

Current Limitations:

- Limited to relatively simple architectures (≤ 10 layers)
- Training epochs are reduced for computational feasibility
- Single-objective scalarization of multi-objective problem

Future Extensions:

- Generalize the approach to different datasets (graph, audio, video, etc)
- Introduce more layers to be considered when constructing the NNs
- Consider advanced components, such as attention, transformers, etc.
- Leverage pre-trained components in search

References I

- [1] M. A. J. Idrissi, H. Ramchoun, Y. Ghanou, and M. Ettaouil, “Genetic algorithm for neural network architecture optimization,” in *2016 3rd International Conference on Logistics Operations Management (GOL)*, May 2016, pp. 1–4. DOI: 10.1109/GOL.2016.7731699. Accessed: Jun. 10, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7731699>.
- [2] M. Carvalho and T. B. Ludermir, “Particle swarm optimization of neural network architectures andWeights,” in *7th International Conference on Hybrid Intelligent Systems (HIS 2007)*, Sep. 2007, pp. 336–339. DOI: 10.1109/HIS.2007.45. Accessed: Jun. 10, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4344074>.

References II

- [3] R. Jamous, A. Hosam, and M. El-Darieby, "Neural network architecture selection using particle swarm optimization technique," *Applied Artificial Intelligence*, vol. 35, no. 15, pp. 1219–1236, Dec. 15, 2021, Publisher: Taylor & Francis _eprint: <https://doi.org/10.1080/08839514.2021.1972251>, ISSN: 0883-9514. DOI: 10.1080/08839514.2021.1972251. Accessed: Jun. 10, 2025. [Online]. Available: <https://doi.org/10.1080/08839514.2021.1972251>.
- [4] C. L. Kuo, E. E. Kuruoglu, and W. K. V. Chan, "Neural network structure optimization by simulated annealing," *Entropy*, vol. 24, no. 3, p. 348, Mar. 2022, Number: 3 Publisher: Multidisciplinary Digital Publishing Institute, ISSN: 1099-4300. DOI: 10.3390/e24030348. Accessed: Jun. 17, 2025. [Online]. Available: <https://www.mdpi.com/1099-4300/24/3/348>.